RFC – CoreBank MEI

Banking-as-a-Service e Governança Patrimonial para o MEI

TIME João Pedro Calsavara · Bruno Cardoso Holanda · Marcel Rebecchi

DATA 11/10/2026 **VERSÃO** 1

Contextualização

Entendendo o problema

O Microempreendedor Individual (MEI) utiliza o mesmo CPF e CNPJ como pessoa natural e empresa, mas precisa manter sua movimentação pessoal e empresarial separada para controle de faturamento e conformidade. O sistema deve cadastrar o MEI e manter contas PF e PJ vinculadas, processar transferências e cobranças comerciais (PIX, boleto e cartão), garantir que o saldo corresponda aos lançamentos contábeis e impedir débitos duplicados por retentativas. Também deve permitir retenção automática para tributos e crédito, sem saldo negativo, além de oferecer CCB e antecipação de recebíveis com trava proporcional ao faturamento. Nesta entrega não estão incluídos cartão físico, outras naturezas jurídicas (ME/EIRELI) ou integração real com SPI/DICT; conectores externos operam em modo simulado.

Explicando a solução de forma macro

A solução é uma API REST em Python/FastAPI com PostgreSQL, organizada em *resource* → *controller* → *repository* → *model*. O *Customer* é a identidade central, vinculada às contas PF e PJ. As movimentações utilizam ledger imutável de partidas dobradas, com valores monetários em `BIGINT` de centavos. Transferências concorrentes bloqueiam as contas em ordem determinística, enquanto operações críticas utilizam `Idempotency-Key`. Cobranças são liquidadas pelo valor bruto, com tarifa debitada na mesma transação, e *Pockets* permitem retenções para tributos e crédito.

- **Conta única com tags:** descartada pelo risco de misturar a movimentação PF/PJ e prejudicar o controle fiscal. Ganharia apenas em simplicidade de banco.
- **Net settlement:** descartado porque a tarifa seria descontada antes do crédito bruto, dificultando a representação da receita. Ganharia em menos lançamentos.
- **FLOAT/DOUBLE:** descartado pelos erros de arredondamento acumulados em operações financeiras. Ganharia apenas em prototipagem simples.
- **Idempotência na transação:** descartada porque não permite cache adequado de respostas e controle de requisições repetidas. Ganharia em simplicidade de schema.

Implementação

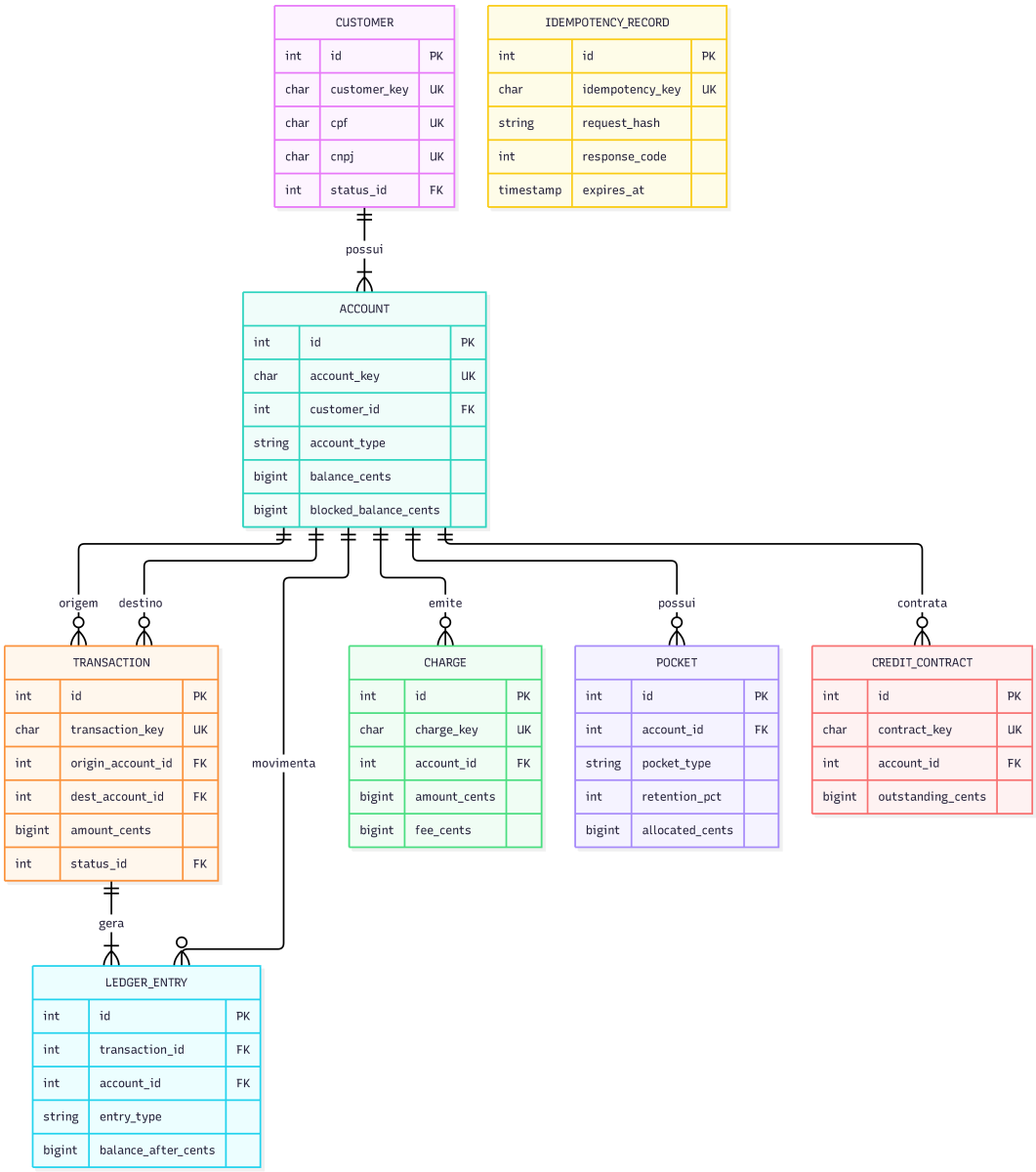
Rotas

Método	Caminho	O que faz	Entrada	Saídas (status e quando)
POST	/customers	Cadastra o MEI e inicia KYC	CPF, CNPJ, e-mail	202 aceito; 400 payload inválido; 409 CPF/CNPJ/e-mail duplicado
POST	/customers/{key}/accounts	Cria contas PF e PJ vinculadas	customer_key	201 criado; 409 cliente não aprovado
PUT	/accounts/{key}/status	Ativa, bloqueia ou encerra conta	status	202 aceito; 409 saldo > 0 no encerramento
POST	/transactions/transfers	Realiza transferência atômica	origem, destino, valor, PIN	201 liquidada; 401 PIN inválido; 409 idempotência; 422 saldo insuficiente
POST	/transactions/{key}/reversals	Realiza estorno contábil	transaction_key	201 estornado; 422 saldo insuficiente na recebedora
GET	/accounts/{key}/statement	Consulta extrato paginado	account_key, paginação	200 lista; 404 conta não encontrada
POST	/accounts/{key}/charges	Cria cobrança comercial	valor, método	201 criada; 422 conta não é PJ

Método	Caminho	O que faz	Entrada	Saídas (status e quando)
POST	/webhooks/payments	Liquida pagamento externo	assinatura, charge	200 liquidado; 401 assinatura inválida
POST	/accounts/{key}/pockets	Cria retenção percentual	tipo, percentual	201 criado; 422 conta não é PJ
POST	/governance/transfer-policies	Define limite PJ → PF	limite mensal	201 configurado; 409 teto excedido
POST	/credit/contracts	Contrata CCB	valor, prazo, PIN	201 desembolsado; 401 PIN inválido; 409 proposta expirada
POST	/credit/anticipations	Antecipa recebíveis	cobranças elegíveis	201 creditado; 422 cobrança inelegível

Todas as escritas críticas utilizam `Idempotency-Key`. A mesma requisição retorna a resposta armazenada; a mesma chave com payload diferente retorna 409.

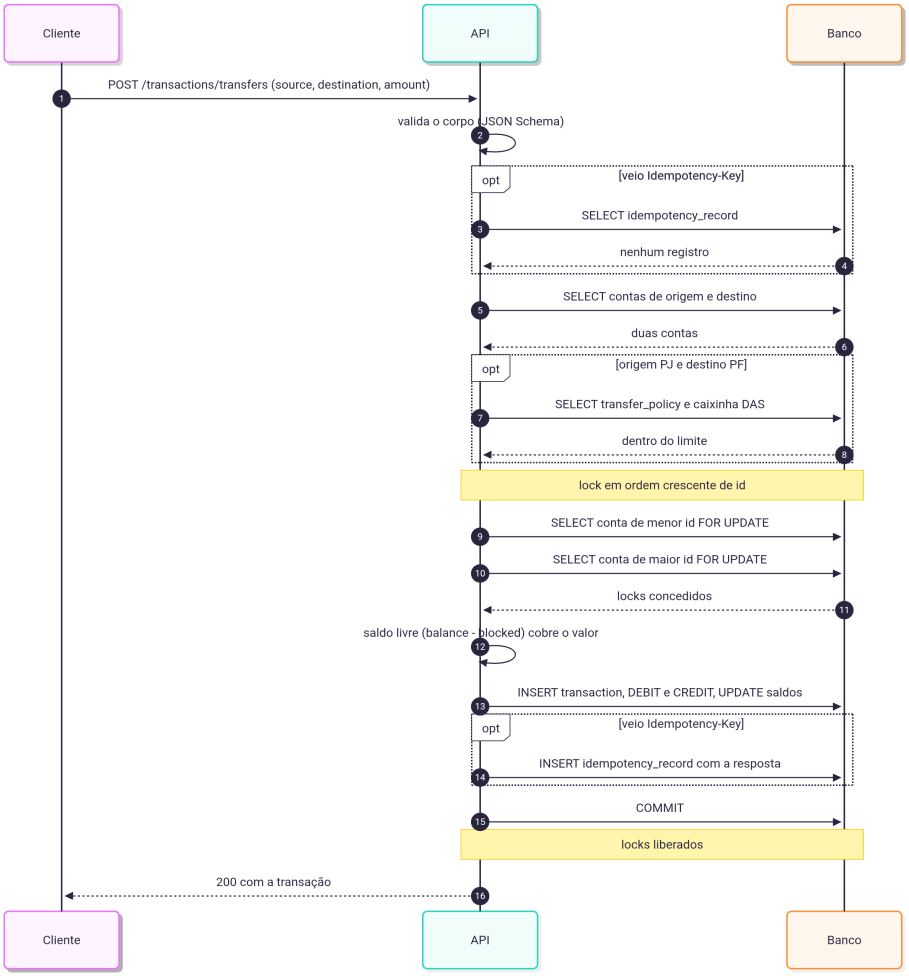
Banco de Dados (Somente diagrama)



Fluxos

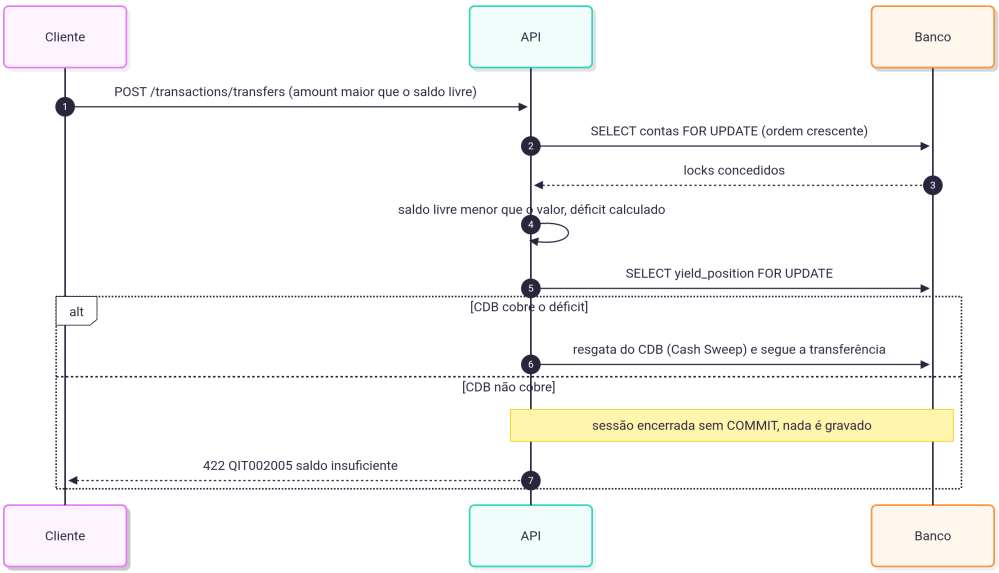
Transferência – caminho feliz

O fluxo representa a validação da requisição, a verificação da idempotência, o bloqueio ordenado das contas, a validação do saldo e a gravação atômica dos lançamentos.



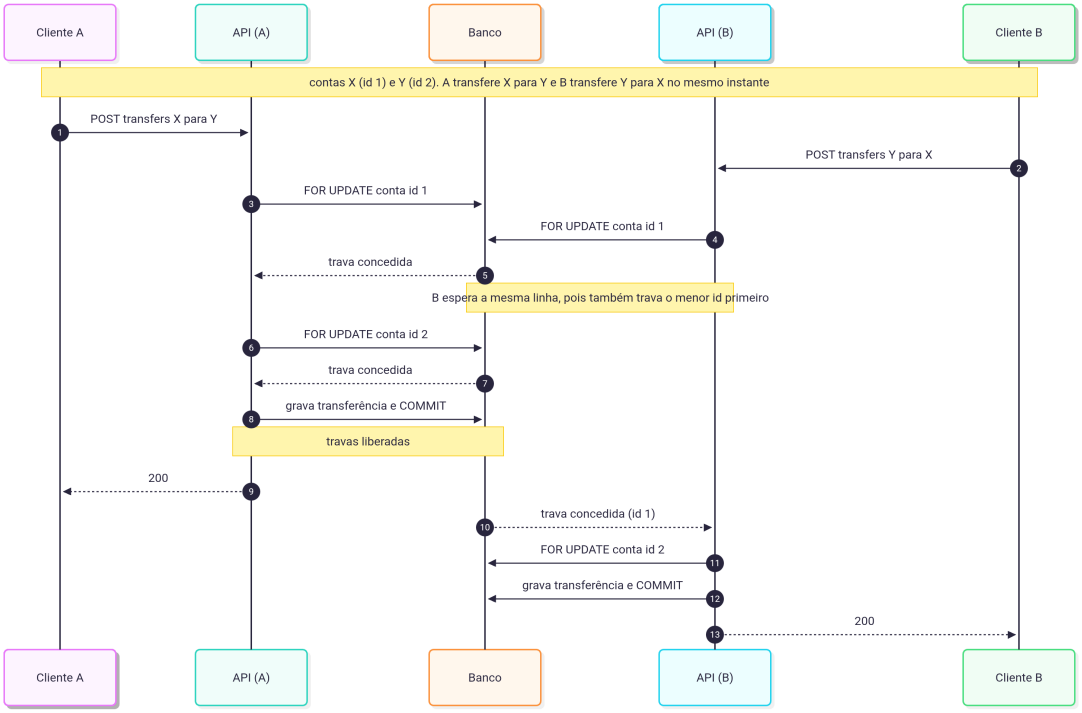
Transferência – saldo insuficiente

Quando o saldo disponível não cobre o valor solicitado, a operação é abortada antes das escritas no ledger e a API retorna o erro 422 (QIT001010). Caso a chave de idempotência seja reutilizada com um payload diferente, a API retorna 409 (QIT001008).



Transferências concorrentes – prevenção de deadlock

Em transferências simultâneas entre as mesmas contas, os IDs internos são ordenados antes da aquisição dos locks. Dessa forma, as operações seguem a mesma ordem de bloqueio, evitando espera circular entre transações.



Principal desafio

Qual é: garantir atomicidade e evitar deadlock quando transferências concorrentes disputam as mesmas contas em sentidos opostos, mantendo também a liquidação de cobranças e tarifas na mesma transação.

Por que é difícil: se cada operação bloquear as contas na ordem recebida, transferências A→B e B→A podem gerar espera circular, causando o erro PostgreSQL 40P01.

Como o desenho resolve: antes de qualquer `SELECT FOR UPDATE`, os IDs internos são ordenados deterministicamente:

```
sorted([id_a, id_b])
```

Assim, todas as transações adquirem os locks na mesma ordem global, eliminando a espera circular.